

Day 12: Advanced Interview Coding Practice

(22-June-2026, Monday)

Session Time: 9:00 AM – 4:30 PM (Lunch Break: 12:30 PM – 1:30 PM, Tea Breaks: 10:30 AM & 3:00 PM)

Session Objectives

By the end of Day 12, learners will be able to:

- Master the **Sliding Window** technique for substring and subarray problems.
- Apply **Hashing** patterns for efficient lookups and frequency counting.
- Understand **Searching** algorithms (Binary Search variants, Search in Rotated Array).
- Implement **Sorting** algorithms and use Java's built-in sorting utilities.
- Recognize common **Competitive Coding Patterns** (Two Pointers, Prefix Sum, Cyclic Sort).
- Solve substring problems like Longest Substring Without Repeating Characters.
- Solve LeetCode: *Longest Substring Without Repeating Characters, Best Time to Buy & Sell Stock, Merge Intervals, Binary Search*.
- Solve HackerRank: *Balanced Brackets, Sparse Arrays, Left Rotation*.
- Design a **Campus Interview Coding Assessment Platform** using these patterns.

Detailed Agenda (Time Slots & Activities)

Time Slot	Activity	Duration
9:00 – 9:15	Recap of Day 11 & Day 12 Overview – Interview Coding Patterns	15 min
9:15 – 10:00	Sliding Window – Fixed Size, Variable Size, Visualization & Examples	45 min
10:00 – 10:45	Hashing Patterns – Frequency Maps, Two-Sum Variants, Prefix Sum	45 min
10:45 – 11:00	Tea Break	15 min
11:00 – 11:30	Searching Algorithms – Binary Search, Search in Rotated Array, Lower/Upper Bound	30 min
11:30 – 12:00	Sorting in Java – Built-in Methods, Custom Comparators	30 min

11:30 – 12:00	Sorting in Java – Built-in Methods, Custom Comparators, QuickSort vs MergeSort	30 min
12:00 – 12:30	Hands-on: Sorting + Binary Search Integration	30 min
12:30 – 1:30	Lunch Break	60 min
1:30 – 2:15	Competitive Coding Patterns – Two Pointers, Prefix Sum, Cyclic Sort	45 min
2:15 – 2:45	Substring Problems – Techniques & Visualizations	30 min
2:45 – 3:00	Tea Break	15 min
3:00 – 3:30	LeetCode Problems Walkthrough (Longest Substring, Best Time, Merge Intervals)	30 min
3:30 – 3:45	HackerRank Problems Walkthrough (Balanced Brackets, Sparse Arrays, Left Rotation)	15 min
3:45 – 4:15	Corporate Use Case: Campus Interview Coding Assessment Platform	30 min
4:15 – 4:30	Homework & Closing	15 min

1. Sliding Window Technique

1.1 What is Sliding Window?

- A technique to process **contiguous subarrays or substrings** efficiently.
- Reduces $O(n^2)$ brute force to $O(n)$ by maintaining a **window** that slides across the array/string.
- Two types: **Fixed Size** and **Variable Size**.

1.2 Fixed Size Sliding Window

- Window size is constant (k).
- Slide the window by one step, update result using previous computation.

Visual Representation

Array: [2, 1, 5, 1, 3, 2]

Window size $k = 3$

Step 1: [2, 1, 5] sum=8 max=8

Step 2: [1, 5, 1] sum=7 max=8 (no change)

Step 3: [5, 1, 3] sum=9 max=9

Step 4: [1, 3, 2] sum=6 max=9

Result: 9

Algorithm (Fixed Size)

1. Compute sum of first k elements.
2. For i from k to n-1:
 - sum = sum - arr[i-k] + arr[i]
 - Update answer with sum

Common Problems

- Maximum sum subarray of size k.
- First negative integer in every window of size k.
- Count occurrences of anagrams.

1.3 Variable Size Sliding Window

- Window size changes dynamically.
- Expand window (add elements) until condition violates, then shrink (remove elements) until condition holds.

Visual Representation – Longest Substring Without Repeating Characters

String: "abcabcbb"

Expected Output: 3 ("abc" or "bca" or "cab")

Step by Step:

i=0, j=0: window="a", length=1, max=1

i=0, j=1: window="ab", length=2, max=2

i=0, j=2: window="abc", length=3, max=3

i=0, j=3: char='a' -> duplicate! shrink from left until 'a' removed
window="bca", length=3, max=3

i=1, j=4: window="bcab" -> char='b' duplicate! shrink
window="cab", length=3, max=3

i=2, j=5: window="cabc" -> char='c' duplicate! shrink
window="abc", length=3, max=3

i=3, j=6: window="abcb" -> char='b' duplicate! shrink
window="cb", length=2, max=3

i=5, j=7: window="cbb" -> char='b' duplicate! shrink
window="b", length=1, max=3

Result: 3

Algorithm (Variable Size)

```
1. Initialize left = 0, maxLen = 0, data structure to track window.
2. For right from 0 to n-1:
  - Add arr[right] to window.
  - While window violates condition:
    - Remove arr[left] from window.
    - left++
  - Update maxLen = max(maxLen, right-left+1)
```

Common Problems

- Longest substring without repeating characters.
- Longest subarray with sum $\leq K$.
- Minimum window substring (hard).

1.4 Sliding Window Pattern Summary

Problem Type	Approach	Data Structure
Fixed window size	Maintain sum/product, slide by one	Variables
Longest substring with unique chars	Expand until duplicate, then shrink	HashSet/HashMap
Longest subarray with sum $\leq K$	Expand until sum $> K$, then shrink	Variables
Minimum window with all chars	Expand until condition met, then shrink to minimize	HashMap + Counter

2. Hashing Techniques

2.1 Why Hashing?

- Provides $O(1)$ average time for lookups, insertions, deletions.
- Ideal for: frequency counting, existence checking, grouping, mapping.

2.2 Hashing Patterns

Pattern 1: Frequency Count (HashMap)

- Count occurrences of elements.
- Example: Majority Element, Valid Anagram, Top K Frequent.

```
Input: [1, 2, 3, 2, 1, 1, 4]
HashMap:
1 -> 3
2 -> 2
3 -> 1
4 -> 1
```

Pattern 2: Two-Sum / Complement Lookup

- For each element, check if complement exists.
- Example: Two Sum, Four Sum, Subarray Sum Equals K.

```
Target = 9, Array = [2, 7, 11, 15]
i=0, complement=7 -> HashMap has 7? No, add 2: {2:0}
i=1, complement=2 -> HashMap has 2? Yes! Found indices (0,1)
```

Pattern 3: Prefix Sum + Hashing

- Track cumulative sum and check if current sum - target exists.
- Example: Subarray Sum Equals K, Contiguous Array.

```
Array: [1, 2, 3], target=3
prefix: 1, 3, 6
Check: prefix[1] - target = 0 (exists) -> subarray [1,2]
Check: prefix[2] - target = 3 (exists) -> subarray [3]
Result: 2 subarrays
```

Pattern 4: Sliding Window + Hashing

- Track character frequencies in window.
- Example: Longest Substring Without Repeating, Anagrams in String.

Pattern 5: Grouping with HashMaps

- Group anagrams, group by length, group by something.

2.3 HashMap vs HashSet – When to Use?

Requirement	Use
Need key-value mapping	<code>HashMap<K,V></code>
Need only existence check	<code>HashSet<E></code>
Need ordered iteration	<code>LinkedHashMap</code> / <code>LinkedHashSet</code>

Need sorted order	TreeMap / TreeSet
Need thread-safe	ConcurrentHashMap

2.4 Hashing Visualization

HashSet Existence Check:
Input: "abcde"
Check 'c': hash("c") → bucket index → found → true

HashMap Frequency:
Array: [a,b,a,c,a,b]
Insert 'a': bucket → value 1
Insert 'a' again: same bucket → value becomes 2

3. Searching Algorithms

3.1 Linear Search (O(n))

- Simple, works on unsorted arrays.
- Not recommended for large datasets.

3.2 Binary Search (O(log n))

- Works on **sorted arrays**.
- Divides search space in half each step.

Standard Binary Search

Input: sorted array [1, 3, 5, 7, 9, 11], target=7
Step 1: left=0, right=5, mid=2 (5) → 5 < 7 → left=3
Step 2: left=3, right=5, mid=4 (9) → 9 > 7 → right=3
Step 3: left=3, right=3, mid=3 (7) → found!

Binary Search Variants

Variant	Use Case
Lower Bound	First occurrence of target ($\geq$ target)
Upper Bound	First element > target
Search Insert Position	Where to insert target

Search in Rotated Array	Find in rotated sorted array
Search in Infinite Array	Exponential search + binary

Visual – Search in Rotated Array

```
Array: [4, 5, 6, 7, 0, 1, 2], target=0
Step 1: mid=3 (7). Left sorted? 4≤7 ≤? 7≤7 true → left sorted.
        target=0 not in [4,7] → search right.
Step 2: left=4, right=6, mid=5 (1). Right sorted? 1≤2 true.
        target=0 in [1,2]? No → search left.
Step 3: left=4, right=4, mid=4 (0) → found!
```

Implementation Checklist

- 1. Ensure array is sorted (or find pivot first).
- 2. Handle edge cases: empty array, target not found.
- 3. Use `left + (right - left) / 2` to avoid overflow.

4. Sorting in Java

4.1 Built-in Sorting Methods

Method	Use Case	Complexity	Features
<code>Arrays.sort(arr)</code>	Sort primitive arrays	$O(n \log n)$	Dual-pivot QuickSort (primitive)
<code>Arrays.sort(objArray)</code>	Sort object arrays	$O(n \log n)$	TimSort (stable, hybrid)
<code>Collections.sort(list)</code>	Sort List	$O(n \log n)$	TimSort
<code>list.sort(comparator)</code>	Sort with custom comparator	$O(n \log n)$	Lambda-friendly
<code>Arrays.parallelSort(arr)</code>	Parallel sorting (large arrays)	$O(n \log n)$	For multi-core

4.2 Sorting Algorithm Comparison

Algorithm	Best	Average	Worst	Space	Stable
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes

MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
TimSort (Java)	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
HeapSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No

4.3 Java's Sort Implementations

Data Type	Sort Algorithm	Why?
Primitive arrays (<code>int[]</code>)	Dual-pivot QuickSort	Faster, less memory (primitive comparison simple)
Object arrays (<code>Integer[]</code>)	TimSort	Stable sort required; hybrid of MergeSort + InsertionSort
Collections (<code>List</code>)	TimSort	Same as above

4.4 Custom Sorting with Comparator

```
// Lambda
list.sort((a, b) -> a.compareTo(b));

// Chained sorting
list.sort(Comparator.comparing(Person::getName)
                    .thenComparing(Person::getAge)
                    .reversed());

// Natural order
Arrays.sort(arr, Comparator.naturalOrder());
```

4.5 Sorting Stability

- **Stable:** Equal elements maintain their original order.
- Java's `Collections.sort` and `Arrays.sort(objects)` are stable.
- `Arrays.sort(primitives)` is **not** stable (but primitives don't have identity).

5. Competitive Coding Patterns

5.1 Pattern 1: Two Pointers

Use Case	Approach
----------	----------

Pair sum in sorted array	left=0, right=n-1, adjust based on sum
Remove duplicates	slow/fast pointer
Trapping rain water	left/right max pointers
Three Sum	Fix one, then two pointers

Visual – Two Sum in Sorted Array

Array: [1, 3, 5, 7, 9], target=12
left=0 (1), right=4 (9): sum=10 < 12 → left++
left=1 (3), right=4 (9): sum=12 == 12 → found!

Algorithm Template

```
int left = 0, right = n - 1;
while (left < right) {
    int sum = arr[left] + arr[right];
    if (sum == target) return true;
    if (sum < target) left++;
    else right--;
}
```

5.2 Pattern 2: Prefix Sum

Use Case	Approach
Subarray sum equals K	Prefix sum + HashMap
Range sum queries	Prefix array (precompute)
Maximum subarray	Kadane's algorithm
Contiguous array (0s/1s)	Prefix sum difference

Visual – Prefix Sum Array

Original: [1, 2, 3, 4]
Prefix: [0, 1, 3, 6, 10]
Range sum [1,3] = prefix[4] - prefix[1] = 10 - 1 = 9

5.3 Pattern 3: Cyclic Sort (for 1..n arrays)

Use Case	Approach

Missing number in 0..n	Swap each number to its correct position
Find all duplicates	Place at correct index, mark visited
Find first missing positive	Cyclic sort + scan

Visual – Cyclic Sort

Array: [3, 1, 5, 2, 4]
 Step 1: arr[0]=3, correct position = 2 (0-based)
 swap arr[0] with arr[2] → [5, 1, 3, 2, 4]
 Step 2: arr[0]=5, correct pos=4, swap → [4, 1, 3, 2, 5]
 Step 3: arr[0]=4, correct pos=3, swap → [2, 1, 3, 4, 5]
 Step 4: arr[0]=2, correct pos=1, swap → [1, 2, 3, 4, 5]
 Now sorted!

5.4 Pattern 4: Substring Problems

Problem Type	Technique
Longest substring without repeating	Sliding Window + HashMap
Minimum window substring	Sliding Window + HashMap
Palindromic substrings	Expand around center / DP
String permutations	Sliding Window + Frequency Map

Substring Pattern Template

```
int left = 0;
HashMap<Character, Integer> map = new HashMap<>();
for (int right = 0; right < n; right++) {
    char c = s.charAt(right);
    map.put(c, map.getOrDefault(c, 0) + 1);

    while (condition violated) {
        char leftChar = s.charAt(left);
        map.put(leftChar, map.get(leftChar) - 1);
        if (map.get(leftChar) == 0) map.remove(leftChar);
        left++;
    }
    updateAnswer(right - left + 1);
}
```

5.5 Popular Problem Categories

Category	Example Problems	Pattern
----------	------------------	---------

Subarrays	Maximum Subarray, Subarray Sum K	Prefix Sum, Kadane
Substrings	Longest Substring, Palindromic	Sliding Window
Two Sum Variants	Two Sum, Three Sum, Four Sum	Two Pointers, Hashing
Rotated Arrays	Rotated Sorted Array Search	Binary Search
Duplicates	Find Duplicates, Missing Numbers	Cyclic Sort, Hashing
Parentheses	Valid Parentheses, Balanced	Stack

6. LeetCode Problems – Approach Discussion

6.1 Longest Substring Without Repeating Characters

- **Problem:** Find length of longest substring without repeating chars.
- **Algorithm:** Sliding Window + HashMap (last seen index).
- **Optimization:** Store last index of each char; jump left pointer directly.
- **Time:** $O(n)$, **Space:** $O(\min(n, \text{alphabet}))$

Visualization

```
s = "abcabcbb"
Start: left=0, right=0, max=0
right=0 'a': last='a'→0, max=1
right=1 'b': last='b'→1, max=2
right=2 'c': last='c'→2, max=3
right=3 'a': last['a']=0 ≥ left? yes → left=1, update last['a']=3, length=3
right=4 'b': last['b']=1 ≥ left? yes → left=2, update last['b']=4, length=3
right=5 'c': last['c']=2 ≥ left? yes → left=3, update last['c']=5, length=3
right=6 'b': last['b']=4 ≥ left? yes → left=5, update last['b']=6, length=2
right=7 'b': last['b']=6 ≥ left? yes → left=7, update last['b']=7, length=1
max = 3
```

6.2 Best Time to Buy & Sell Stock

- **Problem:** Max profit from buying and selling once.
- **Algorithm:** Track min price so far, compute profit at each step.
- **Time:** $O(n)$, **Space:** $O(1)$

Visualization

```
Prices: [7, 1, 5, 3, 6, 4]
min=7, profit=0
i=1, price=1, min=1, profit=0
```

```
i=1: price=1, min=1, profit=0
i=2: price=5, profit=4, min=1
i=3: price=3, profit=2, min=1
i=4: price=6, profit=5, max=5
i=5: price=4, profit=3, max=5
Result: 5 (buy at 1, sell at 6)
```

6.3 Merge Intervals

- **Problem:** Merge overlapping intervals.
- **Algorithm:** Sort by start, then merge.
- **Time:** $O(n \log n)$, **Space:** $O(n)$

Visualization

```
Intervals: [[1,3], [2,6], [8,10], [15,18]]
Sort: [[1,3], [2,6], [8,10], [15,18]]
Start with [1,3]
Next [2,6]:  $2 \leq 3 \rightarrow \text{merge} \rightarrow [1,6]$ 
Next [8,10]:  $8 > 6 \rightarrow \text{add } [8,10], \text{ current}=[8,10]$ 
Next [15,18]:  $15 > 10 \rightarrow \text{add } [15,18], \text{ current}=[15,18]$ 
Result: [[1,6], [8,10], [15,18]]
```

6.4 Binary Search (Standard)

- **Problem:** Find target in sorted array.
- **Algorithm:** Standard binary search.
- **Time:** $O(\log n)$, **Space:** $O(1)$

Visualization

```
Array: [1, 3, 5, 7, 9], target=7
left=0, right=4, mid=2 (5):  $5 < 7 \rightarrow \text{left}=3$ 
left=3, right=4, mid=3 (7): found!
```

7. HackerRank Problems – Approach Discussion

7.1 Balanced Brackets (Stack Pattern)

- **Problem:** Check if brackets are balanced.
- **Algorithm:** Stack for opening brackets; pop on closing.
- **Time:** $O(n)$, **Space:** $O(n)$

Visualization

```
String: "{[()]}"
Stack:
 '[' push →
 '{' push →
 '(' push →
 ')' pop '(' (matches)
 ']' pop '{' (matches)
 '}' pop '[' (matches)
Empty → balanced
```

7.2 Sparse Arrays (HashMap Pattern)

- **Problem:** Given queries, count how many times each query string appears in an array.
- **Algorithm:** Build frequency map of strings; lookup each query.
- **Time:** $O(n + q)$, **Space:** $O(n)$

Visualization

```
Strings: ["ab", "ab", "abc"]
Queries: ["ab", "abc", "bc"]
HashMap: "ab" -> 2, "abc" -> 1
Query "ab": 2
Query "abc": 1
Query "bc": 0
Result: [2, 1, 0]
```

7.3 Left Rotation (Cyclic Pattern)

- **Problem:** Rotate array left by d positions.
- **Approach 1:** Create new array ($O(n)$ space).
- **Approach 2:** Reverse three times ($O(1)$ space).
- **Approach 3:** Use `Collections.rotate()` on list ($O(n)$ time, $O(1)$ space).

Visualization – Reverse Method

```
Array: [1, 2, 3, 4, 5], d=2
Reverse all: [5, 4, 3, 2, 1]
Reverse first n-d = 3: [3, 4, 5, 2, 1]
Reverse last d = 2: [3, 4, 5, 1, 2]
Result: [3, 4, 5, 1, 2]
```

8. Corporate Use Case: Campus Interview Coding Assessment

8.1 Application of Day 12 Topics

Requirement	Pattern/Technique Used
Evaluate substring problems	Sliding Window + Hashing check
Validate subarray sum answers	Prefix Sum validation
Check if candidate sorted correctly	Sorting algorithm verification
Detect inefficient $O(n^2)$ solutions	Monitor runtime, detect patterns
Detect infinite loops	Timeout mechanism
Candidate solution analysis	Compare to optimal approaches ($O(n)$ vs $O(n^2)$)
Visual feedback for sliding window	Show window movement step-by-step
Check for duplicates in solution	Hashing presence check
Validate binary search solutions	Check sorted input requirement
Store problem test cases	HashMap of test cases per problem
Score based on complexity	Analyze loops/recursion patterns

8.2 Pattern Detection in Candidate Solutions

Problem Type	Optimal Pattern	What to Check
Longest Substring	Sliding Window	$O(n)$ solution with HashMap
Subarray Sum K	Prefix Sum + HashMap	$O(n)$ solution, not $O(n^2)$
Merge Intervals	Sorting + Merge	Uses <code>Arrays.sort()</code>
Binary Search	Divide & Conquer	$O(\log n)$ with correct bounds
Two Sum	HashMap	$O(n)$, not $O(n^2)$
Valid Parentheses	Stack	Uses stack, $O(n)$

8.3 Example Assessment Flow

- Candidate submits solution for "Longest Substring Without Repeating".
- Platform runs 5 test cases (size: 100, 1000, 10000, 100000, 1000000).
- If solution $> O(n^2)$ time, times out on large test.
- Platform detects use of `HashSet` or `HashMap` (indicating sliding window).

5. Feedback: "Solution is correct. Time complexity: $O(n)$. Great job!"

9. Quiz & Quick Check (Conceptual)

1. What is the time complexity of sliding window for substring problems?

- (a) $O(n^2)$
- (b) $O(n)$
- (c) $O(n \log n)$
- (d) $O(1)$

2. Which data structure is best for checking duplicate existence in $O(1)$ average time?

- (a) ArrayList
- (b) LinkedList
- (c) HashSet
- (d) TreeSet

3. What is the space complexity of `Arrays.sort()` on primitive arrays?

- (a) $O(1)$
- (b) $O(\log n)$
- (c) $O(n)$
- (d) $O(n \log n)$

4. Which algorithm is used by `Collections.sort()` in Java?

- (a) QuickSort
- (b) MergeSort
- (c) TimSort
- (d) HeapSort

5. In the Longest Substring Without Repeating problem, which data structure is most efficient?

- (a) HashSet
- (b) HashMap (last seen index)
- (c) LinkedList
- (d) Array

Answers: 1-b, 2-c, 3-a (dual-pivot quicksort uses $O(\log n)$ stack space, not $O(1)$), 4-c, 5-b

10. Additional Resources & Cheat Sheets

- **Sliding Window Template** – Fixed and Variable size.
 - **Hashing Patterns Summary** – Frequency, Complement, Prefix, Grouping.
 - **Sorting in Java Cheat Sheet** – Methods and when to use.
 - **Binary Search Template** – Standard, Lower Bound, Upper Bound.
 - **Common Time Complexities:** $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$.
-

11. Closing Summary

Day 12 focused on the most common interview coding patterns. **Sliding Window** enabled $O(n)$ substring/subarray solutions. **Hashing** provided $O(1)$ lookups for frequency and existence problems. **Binary Search** optimized searching in sorted arrays. **Java Sorting** utilities were compared, with TimSort for objects and Dual-Pivot QuickSort for primitives. **Competitive Coding Patterns** (Two Pointers, Prefix Sum, Cyclic Sort) were introduced as essential tools. The Campus Interview Coding Assessment Platform case study tied everything together, showing how these patterns are used to evaluate candidate solutions, detect inefficiencies, and provide feedback.

12. Key Takeaways:

- Sliding Window reduces $O(n^2)$ to $O(n)$ for subarray/substring problems.
- Hashing is the go-to for $O(1)$ lookups.
- Binary Search requires sorted data (or finding pivot).
- Java's built-in sorting is optimized; use it.
- Recognize problem patterns to apply the right technique.

End of Day 12